

ENTERPRISE SECURITY ASSESSMENT LAB

API Security Evidence Documentation Report

JWT Claim Review and Decode-Only Analysis Evidence

Professional Portfolio Edition - Authorized API Security Lab Evidence

Field	Details
Test Area	JWT claim review
Result Type	Token claim analysis
Evidence Report File	07-jwt-decodetoken-analysis-evidence-report.pdf
Testing Phase	Token Claim Review
Tool / Component	Python / PyJWT Decode Without Verification
OWASP / Security Mapping	API2:2023 Broken Authentication
Repository Path	02-api-security/reports/evidence/
Prepared For	GitHub recruiter portfolio documentation

This report documents one API Security evidence row as a standalone professional artifact. It includes the embedded screenshot, what it proves, the methodology context, command/workflow, sanitized output style, risk interpretation, remediation guidance, and publication redaction requirements.

1. Embedded Evidence Screenshot

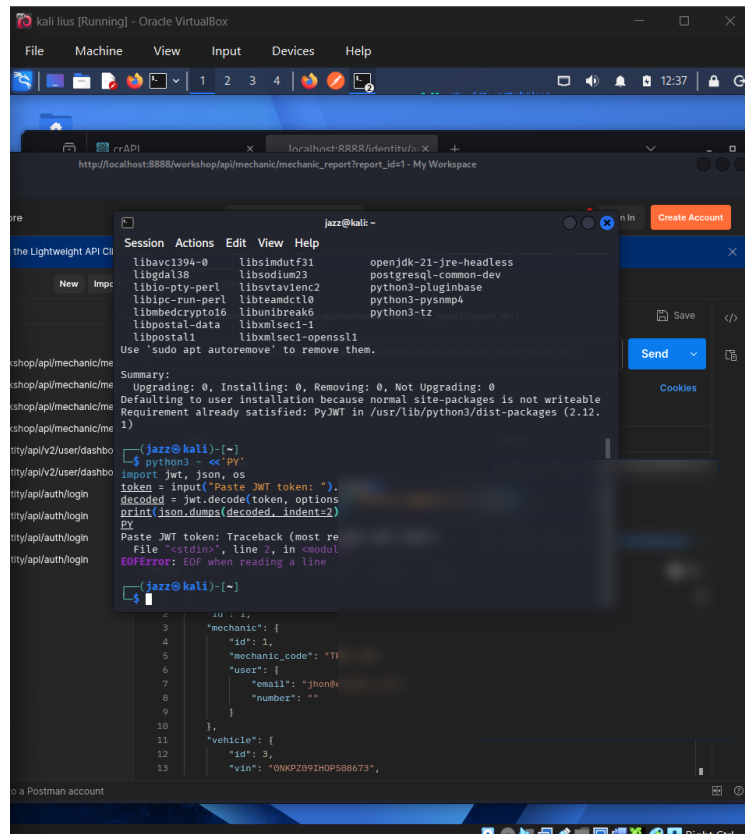


Figure 1 - Embedded screenshot evidence for: JWT claim review

What this evidence proves

The screenshot shows local JWT decoding logic and token claim review. Sensitive token values are blurred.

Evidence handling note

The screenshot has been prepared for GitHub publication. Sensitive fields such as credentials, tokens, JWTs, cookies, personal data, host values, or other secrets are blurred or represented as redacted values where needed.

2. Assessment Context and Methodology Placement

Area	Value
Testing Phase	Token Claim Review
Tool / Component	Python / PyJWT Decode Without Verification
Assessment Objective	Review JWT claims locally without validating the signature in order to understand token content, role claims, user identifiers, and expiry behaviour.
Result Type	Token claim analysis
OWASP / Security Mapping	API2:2023 Broken Authentication

Why this evidence matters

Claim review helps identify whether the API trusts client-side claims, exposes excessive information, or relies on roles that might be targeted in tampering attempts.

Command / workflow used

```
import jwt
decoded = jwt.decode(token, options={"verify_signature": False})
print(json.dumps(decoded, indent=2))
# Review claims only; do not treat decode-only as authentication bypass
```

Sanitized output style

```
Decoded claims reviewed:
sub: <REDACTED>
role: <REVIEWED>
exp/iat: <REVIEWED>
Signature verification: not performed in analysis script
```

3. Professional Interpretation

This evidence row should be interpreted as: **Token claim analysis**.

If an API trusts unverified or tampered JWT claims, attackers may escalate roles or impersonate users. If the token only decodes but the server rejects tampering, this is analysis evidence rather than a vulnerability.

Assessor notes

Clear wording matters: decode-only analysis is not proof of broken authentication.

How a recruiter or reviewer should read this evidence

The value of this evidence is not only the screenshot itself. The value is that it demonstrates a structured API security testing process: controlled lab setup, targeted testing, safe interpretation of results, and clear separation between confirmed vulnerabilities and controls that rejected attack attempts.

Finding interpretation

Where the evidence shows vulnerable behaviour, the report should explain the security impact, the affected object or workflow, and the remediation without publishing sensitive raw values.

4. Risk, Impact, and Remediation Guidance

Risk perspective

If an API trusts unverified or tampered JWT claims, attackers may escalate roles or impersonate users. If the token only decodes but the server rejects tampering, this is analysis evidence rather than a vulnerability.

Recommended remediation / hardening actions

- Always verify signature server-side.
- Never trust decoded claims without validation.
- Use strong signing keys.
- Validate issuer, audience, and expiry.
- Avoid excessive claims.

Validation guidance after remediation

- Repeat the same test workflow after remediation and compare the response behaviour.
- Confirm that sensitive data is no longer exposed in raw API responses.
- Confirm that unauthorized requests return secure error responses such as 401 or 403 where appropriate.
- Confirm that security controls are enforced server-side, not only in the frontend or client collection.

5. GitHub Publication and Redaction Guidance

Sensitive values to redact before public upload

- Full token string.
- User identifiers.
- Roles if private.
- Email addresses.
- Any secret or signing material.

General API evidence redaction checklist

- Blur JWT access tokens, bearer tokens, OAuth codes, cookies, and authorization headers.
- Blur email addresses, phone numbers, user IDs, vehicle IDs, order IDs, and private object identifiers.
- Do not upload Postman environment files containing live variables or secrets.
- Do not upload raw response exports containing personal data or secret-bearing values.
- Use placeholders such as , , and .

GitHub link format for 02-api-security/README.md

[JWT claim review](./reports/evidence/07-jwt-decodetoken-analysis-evidence-report.pdf)

Final classification

This PDF is suitable for a public GitHub portfolio only after the embedded screenshot has been reviewed and sensitive values are confirmed blurred or removed.